
Introduction to Programming in MATLAB

Part 4: Symbolics, Simulink®, Data Structures, File I/O

Outline

(1) Symbolic Math

(2) Simulink

(3) Data Structures

(4) File I/O

Symbolic Math Toolbox

- Symbolics vs. Numerics

	Advantages	Disadvantages
Symbolic	•Analytical solutions •Lets you intuit things about solution form	•Sometimes can't be solved •Can be overly complicated
Numeric	•Always get a solution •Can make solutions accurate •Easy to code	•Hard to extract a deeper understanding •Num. methods sometimes fail •Can take a while to compute

Symbolic Variables

- Symbolic variables are a type, like **double** or **char**
- To make symbolic variables, use **sym**
 - » **a=sym('1/3');**
 - » **b=sym('4/5');**
 - » **mat=sym([1 2;3 4]);**
 - fractions remain as fractions
 - » **c=sym('c','positive');**
 - can add tags to narrow down scope
 - see **help sym** for a list of tags
- Or use **syms**
 - » **syms x y real**
 - shorthand for **x=sym('x','real');** **y=sym('y','real');**

Symbolic Expressions

- Multiply, add, divide expressions

» **d=a*b**

➤ does $1/3 * 4/5 = 4/15$;

d =
4/15

» **expand((a-c)^2) ;**

➤ multiplies out

ans =
 $1/9 - 2/3 * c + c^2$

» **factor(ans)**

➤ factors the expression

ans =
 $1/9 * (3 * c - 1)^2$

» **matInv=inv(mat)**

➤ Computes inverse symbolically

ans =
[-2, 1]
[3/2, -1/2]

Cleaning up Symbolic Statements

» `pretty(ans)`

➤ makes it look nicer


$$\frac{1}{9} - \frac{2}{3}c + c^2$$

» `collect(3*x+4*y-1/3*x^2-x+3/2*y)`

➤ collects terms


$$\text{ans} = 2x + \frac{11}{2}y - \frac{1}{3}x^2$$

» `simplify(cos(x)^2+sin(x)^2)`

➤ simplifies expressions


$$\text{ans} = 1$$

» `subs(c^2,c,5)`

➤ Replaces variables with numbers


$$\text{ans} = 25$$

or expressions. To do multiple substitutions
pass a cell of variable names followed by a cell of values

» `subs(c^2,c,x/7)`


$$\text{ans} = \frac{1}{49}x^2$$

More Symbolic Operations

- We can do symbolics with matrices too

» `mat=sym([a b;c d]);`

» `mat2=mat*[1 3;4 -2];`

➤ compute the product

```
mat2 =  
[ a+4*b, 3*a-2*b]  
[ c+4*d, 3*c-2*d]
```

» `d=det(mat)`

➤ compute the determinant

```
d =  
a*d-b*c
```

» `i=inv(mat)`

➤ find the inverse

```
i =  
[ d/(a*d-b*c), -b/(a*d-b*c)]  
[ -c/(a*d-b*c), a/(a*d-b*c)]
```

- You can access symbolic matrix elements as before

» `i(1,2)`

```
ans =  
-b/(a*d-b*c)
```

Exercise: Symbolics

- The equation of a circle of radius r centered at (a,b) is given by: $(x-a)^2 + (y-b)^2 = r^2$
- Use `solve` to solve this equation for x and then for y
- It's always annoying to integrate by parts. Use `int` to do the following integral symbolically and then compute the value by `subs`tituting 0 for a and 2 for b :

$$\int_a^b x e^x dx$$

Exercise: Symbolics

- The equation of a circle of radius r centered at (a,b) is given by: $(x-a)^2 + (y-b)^2 = r^2$
- Use `solve` to solve this equation for x and then for y

```
» syms a b r x y
```

```
» solve((x-a)^2+(y-b)^2==r^2,x)
```

```
» solve((x-a)^2+(y-b)^2==r^2,y)
```

- It's always annoying to integrate by parts. Use `int` to do the following integral symbolically and then compute the value by `subs` substituting 0 for a and 2 for b :

$$\int_a^b x e^x dx$$

```
» Q=int(x*exp(x),a,b)
```

```
» ans1 =subs(Q,{a,b},{0,2})
```

```
» ans2 = double(ans1)
```

Outline

(1) Symbolic Math

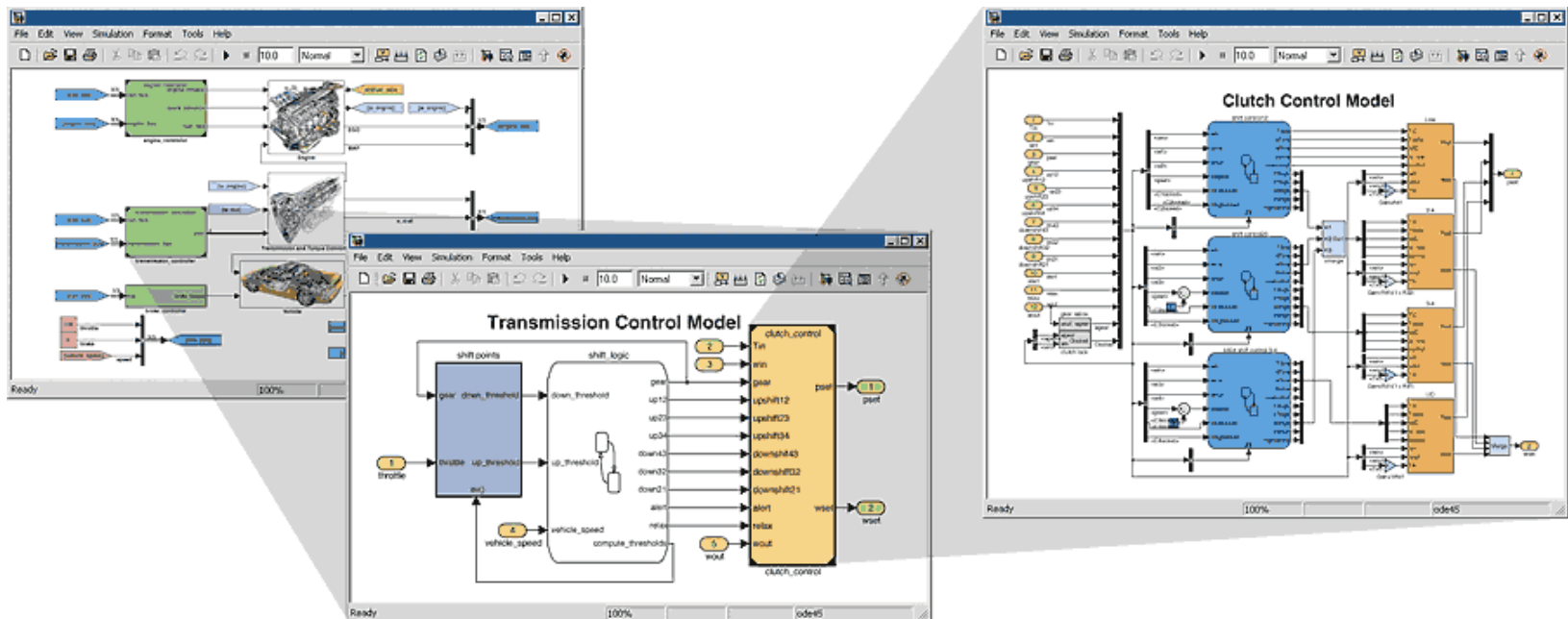
(2) Simulink

(3) Data Structures

(4) File I/O

SIMULINK

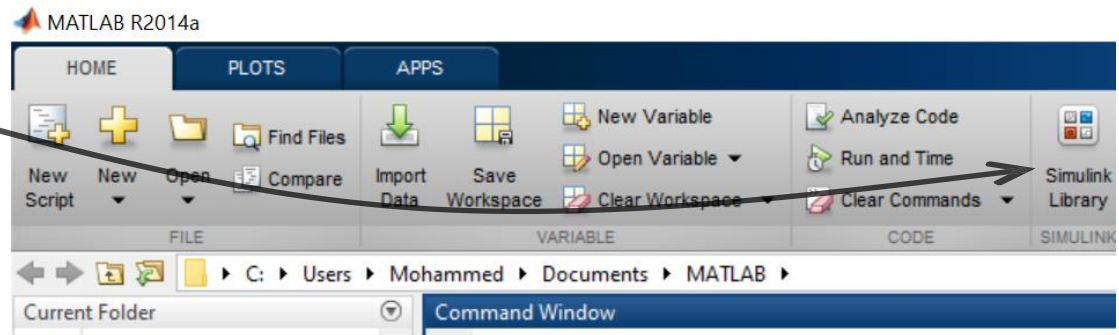
- Interactive graphical environment
- Block diagram based MATLAB add-on environment
- Design, simulate, implement, and test control, signal processing, communications, and other time-varying systems



Courtesy of The MathWorks, Inc. Used with permission.

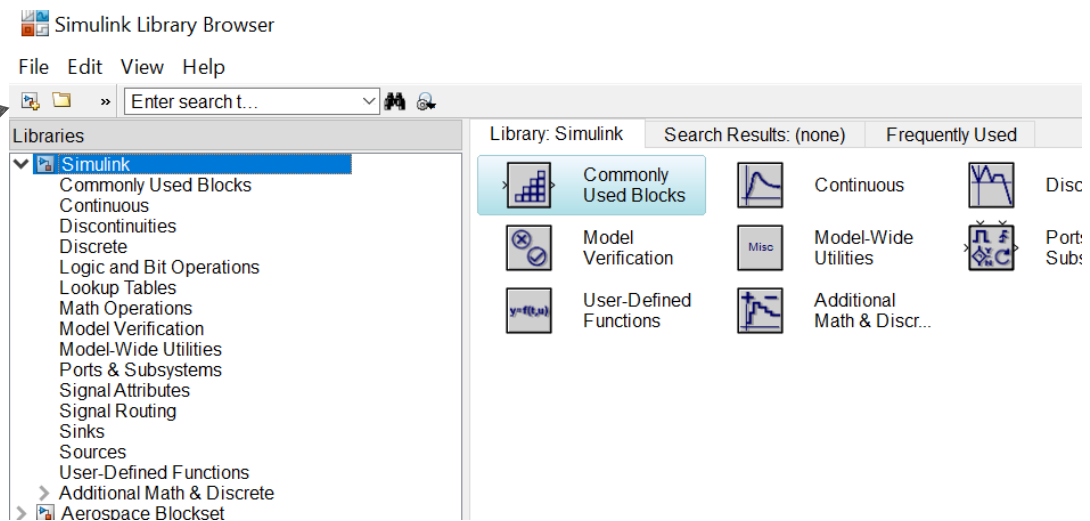
Getting Started

- In MATLAB, Start Simulink



Courtesy of The MathWorks, Inc. Used with permission.

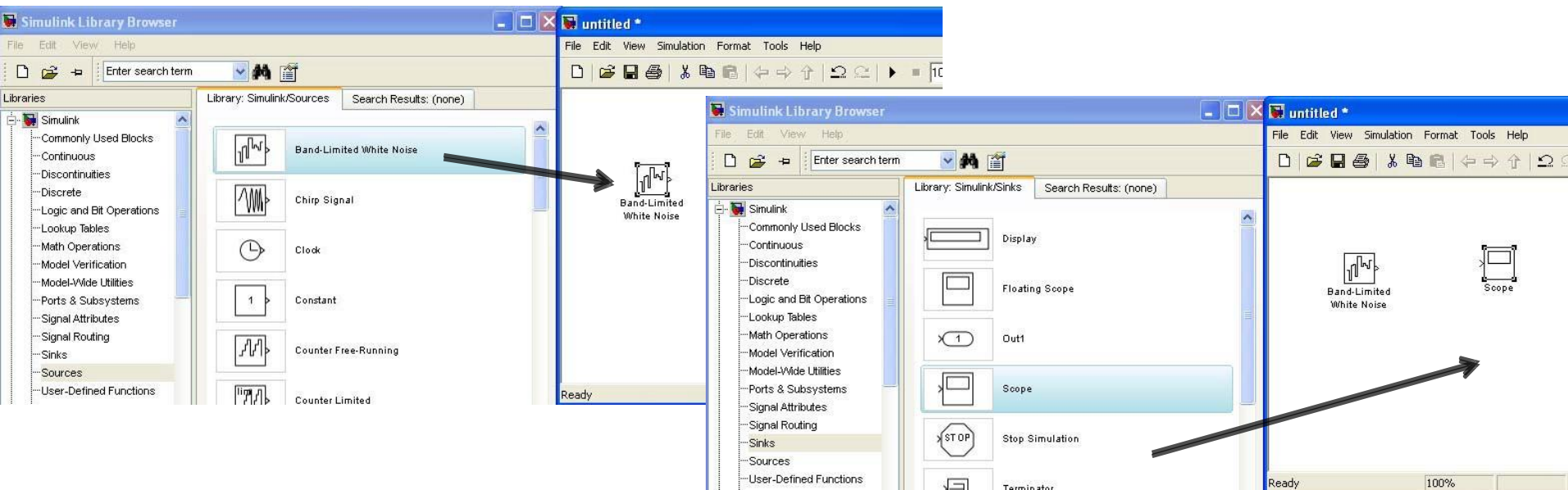
- Create a new Simulink file, similar to how you make a new script



Courtesy of The MathWorks, Inc. Used with permission.

Simulink Library Browser

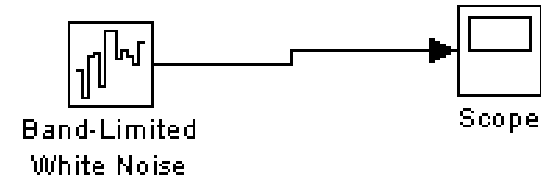
- The **Library Browser** contains various blocks that you can put into your model
- Examine some blocks:
 - Click on a library: **"Sources"**
 - Drag a block into Simulink: **"Band limited white noise"**
 - Visualize the block by going into **"Sinks"**
 - Drag a **"Scope"** into Simulink



Courtesy of The MathWorks, Inc. Used with permission.

Connections

- Click on the carat/arrow on the right of the **band limited white noise** box



- Drag the line to the **scope**
 - You'll get a hint saying you can quickly connect blocks by hitting Ctrl
 - Connections between lines represent signals

- Click the **play** button

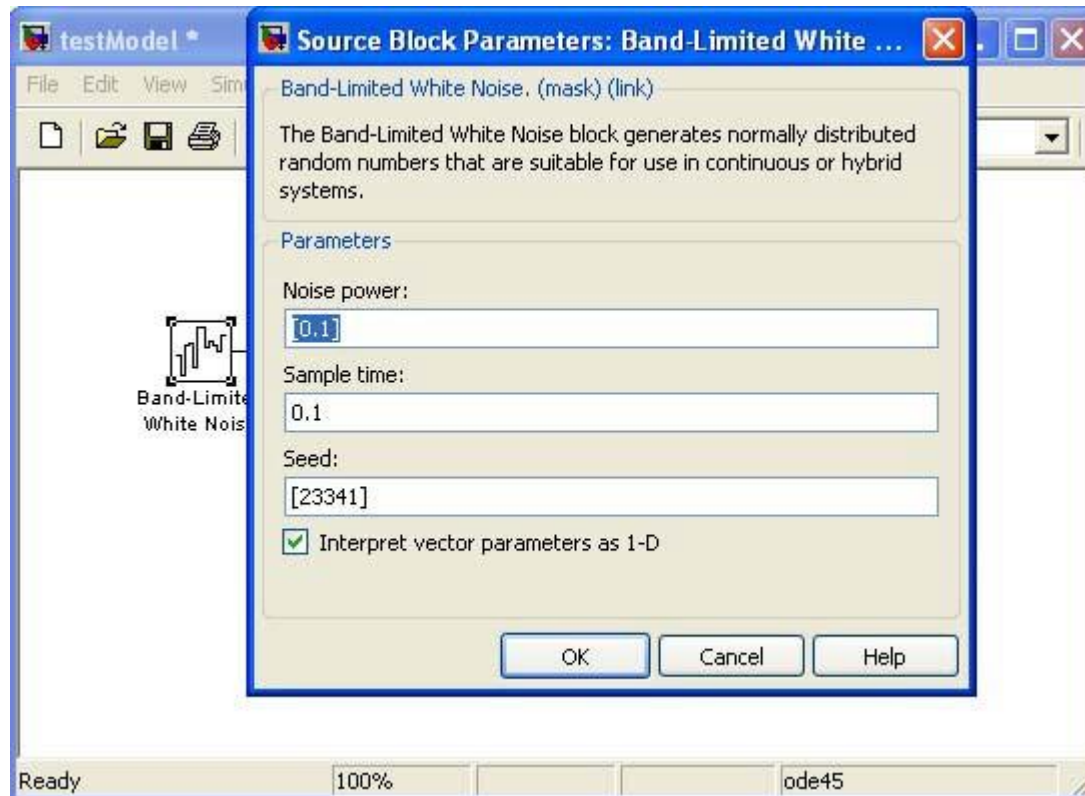


Courtesy of The MathWorks, Inc. Used with permission.

- Double click on the **scope**.
 - This will open up a chart of the variable over the simulation time

Connections, Block Specification

- To split connections, hold down 'Ctrl' when clicking on a connection, and drag it to the target block; or drag backwards from the target block
- To modify properties of a block, double-click it and fill in the property values.



Behind the curtain

- Go to "Simulation"->"Configuration Parameters" at the top menu

See ode45? Change the solver type here

The screenshot shows the 'Configuration Parameters' dialog box for a simulation. It is divided into several sections: 'Simulation time', 'Solver options', 'Tasking and sample time options', and 'Zero crossing options'. Red arrows point from the text 'See ode45? Change the solver type here' to the 'Type' dropdown menu (set to 'Variable-step') and the 'Solver' dropdown menu (set to 'ode45 (Dormand-Prince)').

Section	Parameter	Value
Simulation time	Start time	0.0
	Stop time	10.0
Solver options	Type	Variable-step
	Solver	ode45 (Dormand-Prince)
	Max step size	auto
	Min step size	auto
	Initial step size	auto
	Consecutive min step size violations allowed	1
	States shape preservation	Disable all
Tasking and sample time options	Tasking mode for periodic sample times	Auto
	☐ Automatically handle rate transition for data transfer	
	☐ Higher priority value indicates higher task priority	
Zero crossing options	Zero crossing control	Use local settings
	Zero crossing location algorithm	Non-adaptive
	Consecutive zero crossings relative tolerance	10*128*eps
	Zero crossing location threshold	auto
	Number of consecutive zero crossings allowed	1000

Exercise: Simulink

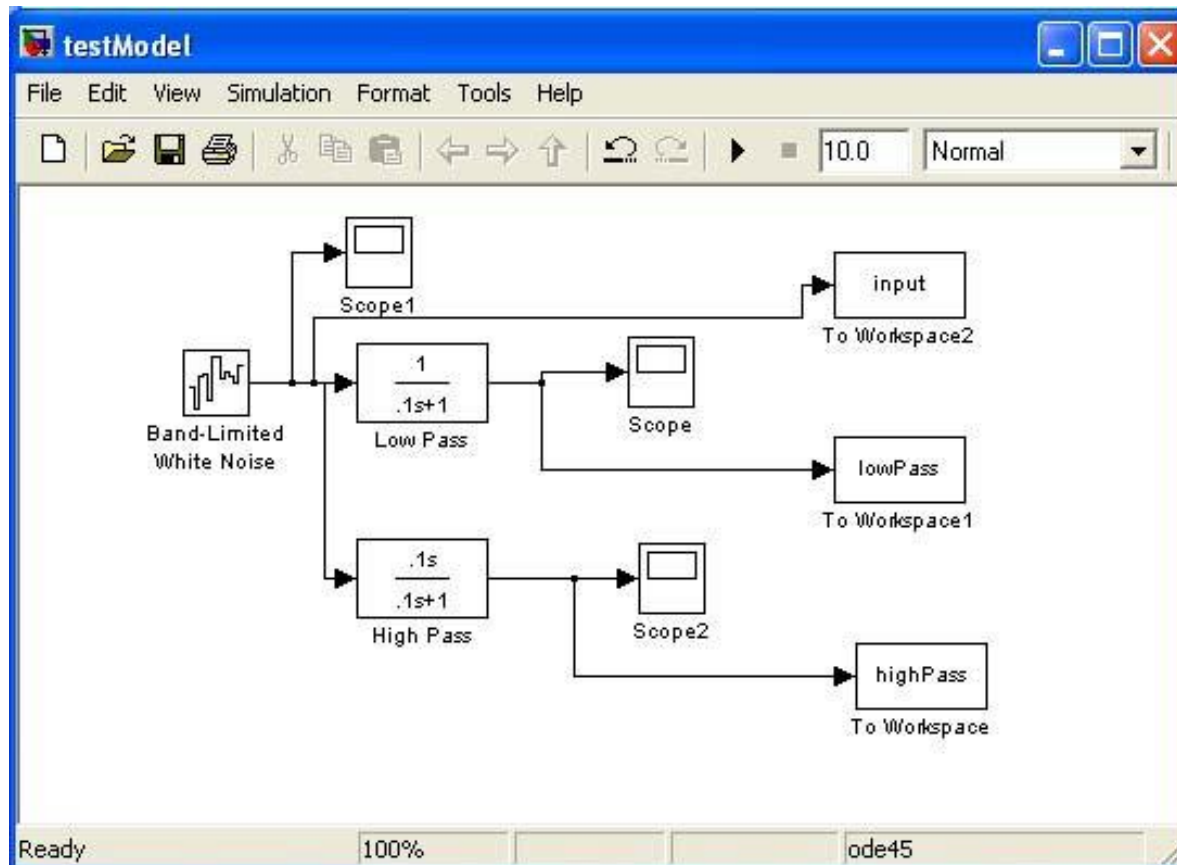
- Take your white noise signal, and split it into high frequency and low frequency components. Use the **Transfer Function** block from **Continuous** and use these transfer functions:

$$LP = \frac{1}{0.1s + 1} \quad HP = \frac{0.1s}{0.1s + 1}$$

- Hook up scopes to the input and the two outputs
- Send the two outputs to the workspace by using the **to Workspace** block from **Sink**

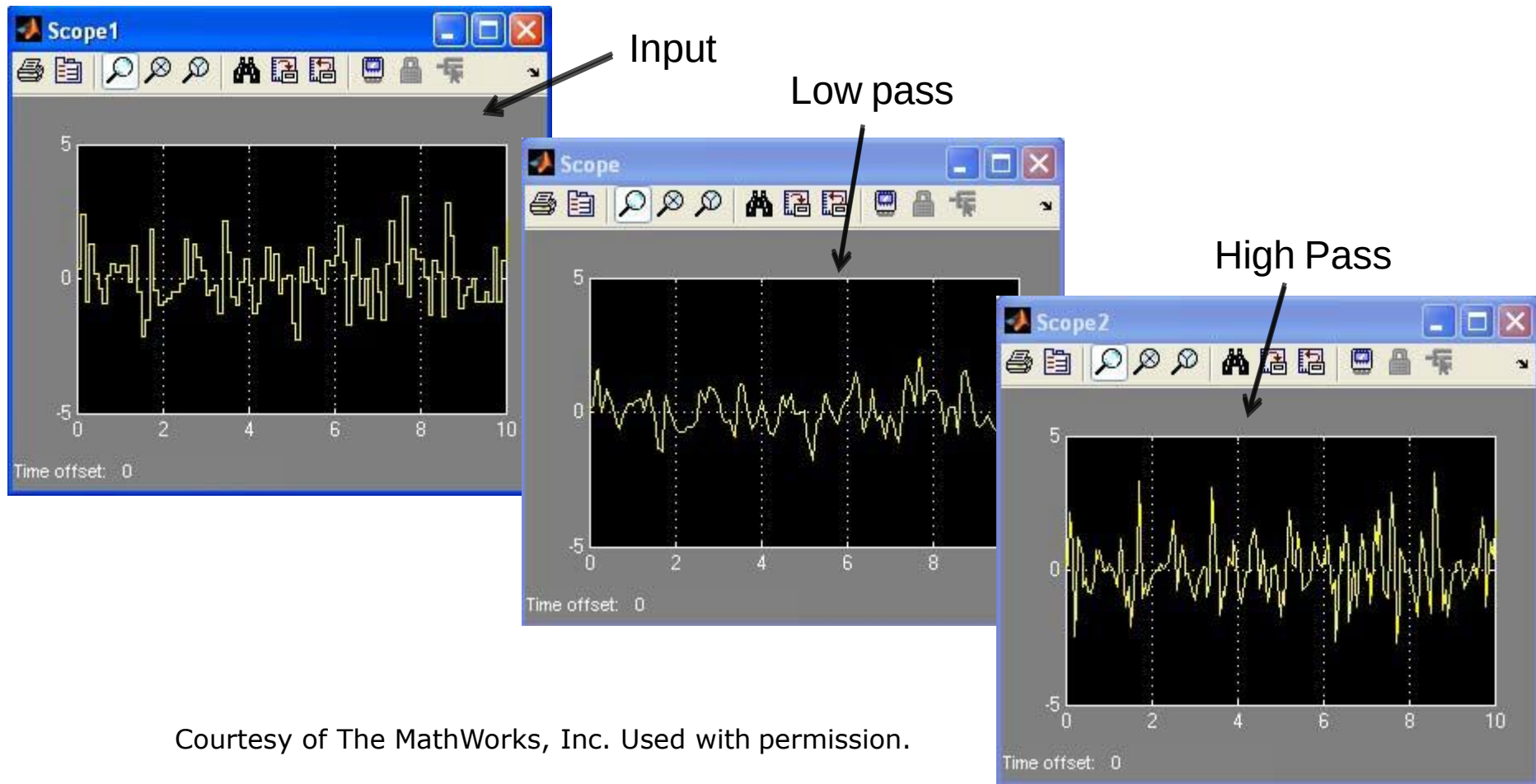
Exercise: Simulink

- The diagram should look like this. To change the **transfer function** parameters, double click the blocks and specify the numerator and denominator as polynomials in s (remember how we defined polynomial vectors before)



Exercise: Simulink

- After running the simulation, double-clicking the scopes will show:



Courtesy of The MathWorks, Inc. Used with permission.

Toolboxes

- Math
 - Takes the signal and performs a math operation
 - » Add, subtract, round, multiply, gain, angle
- Continuous
 - Adds differential equations to the system
 - » Integrals, Derivatives, Transfer Functions, State Space
- Discontinuities
 - Adds nonlinearities to your system
- Discrete
 - Simulates discrete difference equations
 - Useful for digital systems

Building systems

- Sources
 - » Step input, white noise, custom input, sine wave, ramp input,
 - Provides input to your system
- Sinks
 - » Scope: Outputs to plot
 - » simout: Outputs to a MATLAB vector on workspace
 - » MATLAB mat file

Outline

(1) Symbolic Math

(2) Simulink

(3) Data Structures

(4) File I/O

Advanced Data Structures

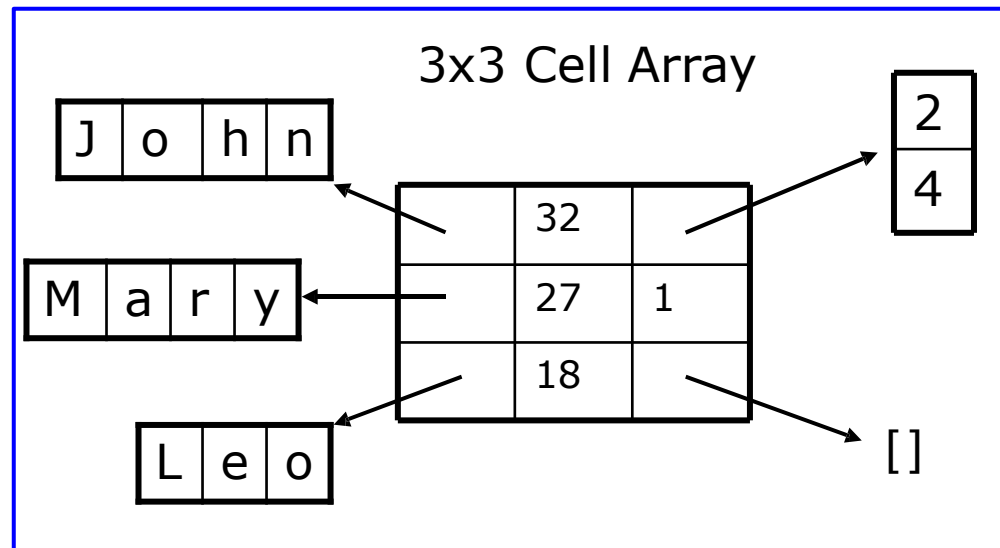
- We have used 2D matrices
 - Can have n-dimensions
 - Every element must be the same type (ex. integers, doubles, characters...)
 - Matrices are space-efficient and convenient for calculation
 - Large matrices with many zeros can be made sparse:
 - » `a=zeros(100); a(1,3)=10;a(21,5)=pi; b=sparse(a);`
- Sometimes, more complex data structures are more appropriate
 - **Cell array**: it's like an array, but elements don't have to be the same type
 - **Structs**: can bundle variable names and values into one structure
 - Like object oriented programming in MATLAB

Cells: organization

- A cell is just like a matrix, but each field can contain anything (even other matrices):

3x3 Matrix

1.2	-3	5.5
-2.4	15	-10
7.8	-1.1	4



- One cell can contain people's names, ages, and the ages of their children
- To do the same with matrices, you would need 3 variables and padding

Cells: initialization

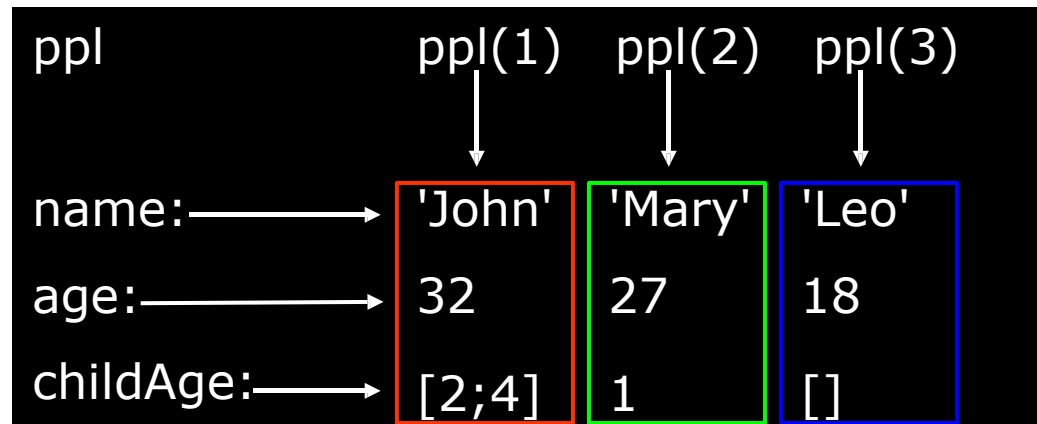
- To initialize a cell, specify the size
 - » `a=cell(3,10);`
 - a will be a cell with 3 rows and 10 columns
- or do it manually, with curly braces {}
 - » `c={'hello world',[1 5 6 2],rand(3,2)};`
 - c is a cell with 1 row and 3 columns
- Each element of a cell can be anything
- To access a cell element, use curly braces {}
 - » `a{1,1}=[1 3 4 -10];`
 - » `a{2,1}='hello world 2';`
 - » `a{1,2}=c{3};`

Structs

- Structs allow you to name and bundle relevant variables
 - Like C-structs, which are objects with fields
- To initialize an empty struct:
 - » `s=struct([]);`
 - `size(s)` will be 1x1
 - initialization is optional but is recommended when using large structs
- To add fields
 - » `s.name = 'Jack Bauer';`
 - » `s.scores = [95 98 67];`
 - » `s.year = 'G3';`
 - Fields can be anything: matrix, cell, even struct
 - Useful for keeping variables together
- For more information, see **doc struct**

Struct Arrays

- To initialize a struct array, give field, values pairs
 - » `ppl=struct('name',{'John','Mary','Leo'},...
'age',{32,27,18},'childAge',{[2;4],1,[]});`
 - `size(s2)=1x3`
 - every cell must have the same size
 - » `person=ppl(2);`
 - person is now a struct with fields name, age, children
 - the values of the fields are the second index into each cell
 - » `person.name`
 - returns 'Mary'
 - » `ppl(1).age`
 - returns 32



Structs: access

- To access 1x1 struct fields, give name of the field
 - » `stu=s.name;`
 - » `scor=s.scores;`
 - 1x1 structs are useful when passing many variables to a function. put them all in a struct, and pass the struct
- To access nx1 struct arrays, use indices
 - » `person=ppl(2);`
 - person is a struct with name, age, and child age
 - » `personName=ppl(2).name;`
 - personName is 'Mary'
 - » `a=[ppl.age];`
 - a is a 1x3 vector of the ages; this may not always work, the vectors must be able to be concatenated.

Exercise: Cells

- Write a script called `sentGen`
- Make a 3x2 cell, and put three **names** into the first column, and **adjectives** into the second column
- Pick two random integers (values 1 to 3)
- Display a sentence of the form '[name] is [adjective].'
- Run the script a few times

Exercise: Cells

- Write a script called `sentGen`
- Make a 3x2 cell, and put three **names** into the first column, and **adjectives** into the second column
- Pick two random integers (values 1 to 3)
- Display a sentence of the form '[name] is [adjective].'
- Run the script a few times

```
» c=cell(3,2);  
» c{1,1}='John' ;c{2,1}='Mary-Sue' ;c{3,1}='Gomer' ;  
» c{1,2}='smart' ;c{2,2}='blonde' ;c{3,2}='hot'  
» r1=ceil(rand*3);r2=ceil(rand*3);  
» disp([ c{r1,1}, ' is ', c{r2,2}, '.' ]);
```

Outline

(1) Symbolic Math

(2) Simulink

(3) Data Structures

(4) File I/O

Importing Data

- MATLAB is a great environment for processing data. If you have a text file with some data:

```
jane joe jimmy
10 11 12
5 4 2
5 6 4
```

- To import data from files on your hard drive, use `importdata`

```
» a=importdata('textFile.txt');
```

➤ a is a struct with `data`, `textdata`, and `colheaders` fields

```
a =
    data: [3x3 double]
 textdata: {'jane'  'joe'  'jimmy'}
colheaders: {'jane'  'joe'  'jimmy'}
```

```
» x=a.data;
```

```
» names=a.colheaders;
```


Importing Data

- With `importdata`, you can also specify delimiters. For example, for comma separated values, use:
 - » `a=importdata('filename', ',');`
 - The second argument tells matlab that the tokens of interest are separated by commas or spaces
- `importdata` is very robust, but sometimes it can have trouble. To read files with more control, use `fscanf` (similar to C/Java), `textread`, `textscan`. See `help` or `doc` for information on how to use these functions

Writing Excel Files

- MATLAB contains specific functions for reading and writing Microsoft Excel files
- To write a matrix to an Excel file, use `xlswrite`
 - » `[s,m]=xlswrite('randomNumbers',rand(10,4),...
'Sheet1'); % we specify the sheet name`
- You can also write a cell array if you have mixed data:
 - » `C={'hello','goodbye';10,-2;-3,4};`
 - » `[s,m]=xlswrite('randomNumbers',C,'mixedData');`
- `s` and `m` contain the 'success' and 'message' output of the write command
- See `doc xlswrite` for more usage options

Reading Excel Files

- Reading excel files is equally easy
- To read from an Excel file, use `xlsread`
 - » `[num,txt,raw]=xlsread('randomNumbers.xls');`
 - Reads the first sheet
 - `num` contains numbers, `txt` contains strings, `raw` is the entire cell array containing everything
 - » `[num,txt,raw]=xlsread('randomNumbers.xls',... 'mixedData');`
 - Reads the **mixedData** sheet
 - » `[num,txt,raw]=xlsread('randomNumbers.xls',-1);`
 - Opens the file in an Excel window and lets you click on the data you want!
- See `doc xlsread` for even more fancy options